

DATABASE DESIGN FOR ECOMMERCE

DBMS Lab Project Report

TABLE OF CONTENTS

1. Introduction
2. Database Schema
3. SQL Queries and Results
 - 3.1 Basic Join Operations
 - 3.2 Outer Join Operations
 - 3.3 Aggregate Functions and Grouping
 - 3.4 Views
 - 3.5 Transactions
4. Conclusion

1. INTRODUCTION

This report presents a comprehensive database design for an E-commerce system. The database implements a relational model using SQL, demonstrating fundamental database concepts including table creation, relationships, joins, aggregations, views, and transactions.

The E-commerce database consists of four main tables: Customers, Products, Orders, and Order Items. The design follows normalization principles and implements referential integrity through foreign key constraints.

2. DATABASE SCHEMA

The database consists of four interconnected tables:

2.1 Customers Table

Column	Data Type
customer_id (PK)	INT
first_name	VARCHAR(50)
last_name	VARCHAR(50)
email	VARCHAR(100)
city	VARCHAR(50)
country	VARCHAR(50)
join_date	DATE

2.2 Products Table

Column	Data Type
product_id (PK)	INT
product_name	VARCHAR(100)
category	VARCHAR(50)
price	DECIMAL(10, 2)
stock_quantity	INT

2.3 Orders Table

Column	Data Type
order_id (PK)	INT
customer_id (FK)	INT
order_date	DATE
total_amount	DECIMAL(10, 2)
status	VARCHAR(20)

2.4 Order Items Table

Column	Data Type
order_item_id (PK)	INT
order_id (FK)	INT
product_id (FK)	INT
quantity	INT
price_per_unit	DECIMAL(10, 2)

DATABASE DESIGN

This database is designed using draw.SQL:

Please find original design here: [E-Commerce database design in draw.SQL](#)

customers

 customer_id	int
first_name	varchar
last_name	varchar
email	varchar
city	varchar
country	varchar
join_date	date

orders

 order_id	int
customer_id	int
order_date	date
total_amount	decimal
status	varchar

products

 product_id	int
product_name	varchar
category	varchar
price_detail	decimal
stock_quantity	int

order_items

 order_item_id	bigint
order_id	int
product_id	int
quantity	int
price_per_unit	decimal

 drawSQL

3. SQL QUERIES AND RESULTS

3.1 Basic Join Operations

Query 1: Customer Orders

Display customer names along with their order details.

```
SELECT first_name, last_name, order_id, order_date, total_amount, status FROM customers INNER JOIN orders  
ON customers.customer_id = orders.customer_id;
```

Result: 15 rows returned showing customer order information

Query 2: Products Sold

Show all products that have been sold with details.

```
SELECT product_name, category, price, quantity, order_date FROM orders INNER JOIN ( SELECT  
order_items.order_id, product_name, category, price, quantity FROM products INNER JOIN order_items ON  
products.product_id = order_items.product_id ) as prods_order ON prods_order.order_id = orders.order_id;
```

Query 3: Customer Product Purchases

Find which customers ordered which products.

```
SELECT first_name, last_name, product_name, quantity FROM customers INNER JOIN ( SELECT customer_id,  
product_name, quantity FROM orders INNER JOIN ( SELECT order_id, product_name, quantity FROM  
products INNER JOIN order_items ON products.product_id = order_items.product_id ) AS mid ON  
orders.order_id = mid.order_id ) AS final ON customers.customer_id = final.customer_id;
```

3.2 Outer Join Operations

Query 4: All Products with Order Statistics

Show all products including those never ordered, with total times ordered and quantity sold.

```
SELECT tab.product_id, COUNT(tab.order_item_id) as no_of_times, COUNT(tab.quantity) as total_quantity
FROM ( SELECT products.product_id, products.product_name, order_items.order_item_id,
order_items.quantity FROM products LEFT JOIN order_items ON products.product_id =
order_items.product_id) as tabGROUP BY tab.product_id;
```

Query 5: All Customers with Purchase Statistics

Display all customers with their total orders and amount spent.

```
SELECT tab.customer_id, COUNT(tab.order_id) as total_orders_placed, SUM(tab.total_amount) as total_spent
FROM ( SELECT customers.customer_id, orders.order_id, orders.total_amount FROM orders LEFT JOIN
customers ON customers.customer_id = orders.customer_id) as tabGROUP BY tab.customer_id;
```

3.3 Aggregate Functions and Grouping

Query 6: Customers by Country

Count customers from each country.

```
SELECT country, COUNT(customer_id) as total_customers FROM customers GROUP BY country;
```

Query 8: Products by Category with Average Price

Count products in each category and show average price.

```
SELECT category, AVG(price) as avg_price_per_category FROM products GROUP BY category;
```

Query 9: Orders per Customer

Find how many orders each customer has placed.

```
SELECT customers.first_name, customers.last_name, tab1.total_orders_placed FROM customers, (SELECT tab.customer_id, COUNT(order_id) as total_orders_placed FROM (SELECT customers.customer_id, customers.first_name, customers.last_name, orders.order_id FROM customers INNER JOIN orders ON customers.customer_id = orders.customer_id ) AS tab GROUP BY tab.customer_id) as tab1 WHERE tab1.customer_id = customers.customer_id;
```

Query 10: High-Value Customers

Calculate revenue and average order value for customers who spent over \$500.

```
SELECT tab1.customer_id, COUNT(tab1.order_id) AS Total_orders, SUM(tab1.total_amount) AS Total_revenue, AVG(tab1.total_amount) AS avg_order_value FROM ( SELECT customers.customer_id, orders.order_id, orders.total_amount FROM customers JOIN orders ON customers.customer_id = orders.customer_id) AS tab1 GROUP BY tab1.customer_id HAVING SUM(tab1.total_amount) > 500;
```

Query 12: Top 5 Best-Selling Products

Find the top 5 products by total revenue.

```
SELECT DISTINCT TOP(5) tab3.product_id, tab3.Product_revenue FROM ( SELECT tab2.product_id, tab2.Total_qnty, order_items.price_per_unit, (tab2.Total_qnty * order_items.price_per_unit) AS Product_revenue FROM order_items LEFT JOIN ( SELECT tab.product_id, COUNT(tab.quantity) as Total_qnty FROM (SELECT products.product_id, order_items.quantity FROM products INNER JOIN order_items ON products.product_id = order_items.product_id ) AS tab GROUP BY tab.product_id ) AS tab2 ON order_items.product_id = tab2.product_id) AS tab3 ORDER BY tab3.Product_revenue DESC;
```


3.4 Views

Query 13: Create Customer Order Summary View

Create a view showing customer order summaries.

```
CREATE VIEW customer_order_summary AS SELECT customers.customer_id, first_name, last_name, email,
country, Total_orders, Total_spent FROM ( SELECT customer_id, COUNT(order_id) AS Total_orders,
SUM(total_amount) AS Total_spent FROM orders GROUP BY customer_id) AS tab LEFT JOIN customers ON
customers.customer_id = tab.customer_id; SELECT * FROM customer_order_summary;
```

Query 14: Query View for High Spenders

Use the view to find customers who spent more than \$1000.

```
SELECT first_name, last_name, Total_spent FROM customer_order_summary WHERE Total_spent > 1000;
```

3.5 Transactions

Query 15: Successful Transaction

Insert a new customer with order and commit the transaction.

```
BEGIN TRANSACTION add_customer_data;INSERT INTO customersVALUES (11, 'Alex', 'Turner',  
'alex.turner@gmail.com', 'Manchester', 'UK', '2024-02-12');INSERT INTO orders (order_id, total_amount,  
status, order_date)VALUES(16, 299.99, 'Processing', '2024-02-12');INSERT INTO order_items (order_item_id,  
quantity, price_per_unit)VALUES (99, 10, 69);COMMIT;
```

Query 16: Failed Transaction with Rollback

Attempt to insert an order for non-existent customer and rollback.

```
BEGIN TRANSACTION fail_attempt;INSERT INTO ordersVALUES (99, 777, '2020-02-20', '1000', 'Processing');  
ROLLBACK;
```

Query 17: Multi-Update Transaction

Update order statuses in a transaction and commit.

```
BEGIN TRANSACTION change_status;UPDATE ordersSET status = 'Shipped'WHERE status =  
'Processing';UPDATE ordersSET status = 'Delivered'WHERE status = 'Shipped';COMMIT;
```

Project link:

Github Repository: [E-commerce database design project link](#)

4. CONCLUSION

This project presents a database for an E-commerce system using SQL. The database has four tables that store information about customers, products, orders, and order items. The design follows normalization principles and uses foreign keys to maintain relationships between tables.

The queries cover different SQL operations. Inner joins combine data from multiple tables. Left joins retrieve all records even when some tables have no matching data. Aggregate functions like COUNT, SUM, and AVG calculate statistics from grouped data. The HAVING clause filters these grouped results. Views store complex queries for repeated use. Transactions group multiple operations together to maintain data consistency.

The queries address practical business needs such as tracking customer purchases, analyzing product sales, calculating revenue, and managing order status. The database uses proper foreign key constraints to prevent invalid data entries.